

전체 코드 로직 & 변수 정리

주제: 도로 빙결 위험지역 분석을 통한 열선 우선설치 도로 제시

변수:

- **경사도(고도):** 정거장 노드를 중심으로 도로 구간의 경사도를 분석.
- **인구이동(승하차 데이터):** 승하차 데이터(버스 승하차 위치와 인구 이동)를 기반으로 도로의 유동인구 분석.
- **결빙 위험도:** 결빙 위험도를 분석하여 위험이 큰 도로 구간을 우선적으로 추출.
- **열선 설치 유무:** 이미 열선이 설치된 도로 구간을 제외하고, 새로운 열선 설치가 필요한 구간을 도출.

절차:

1. **승하차 데이터 분석:** 정거장과 정거장 사이의 도로 구간에 대한 유동인구 분석.
2. **경사도 분석:** 정거장 노드를 중심으로 도로의 경사도를 분석.
3. **결빙 위험도 분석:** 결빙 위험도는 특정 반경(예: 0.5km) 내 도로에 대해 위험도를 산출.
4. **도로 구간 분석:** 경사도, 결빙 위험도, 승하차 데이터를 결합하여 위험도가 높은 도로 구간을 추출.
5. **열선 설치 유무 확인:** 이미 열선이 설치된 도로는 제외하고, 우선 설치해야 할 도로 구간을 도출.

1. 승하차 데이터 분석

1. 정류소 정보 병합

- **로직:**
 - 서울시의 정류소 정보와 버스 정류소 위치 정보를 병합합니다.
 - 광진구에 위치한 정류소만 필터링하여 해당 데이터를 추출합니다.
 - 위치 정보(경도, 위도)를 포함한 CSV 파일과 엑셀 파일을 병합하여, 정류소에 대한 다양한 정보를 포함한 데이터를 생성합니다.
- **결과:** 광진구에 위치한 정류소에 대한 정보가 포함된 데이터프레임이 생성됩니다.

2. 수동 좌표 보완

- **로직:**
 - 일부 정류소의 좌표는 수동으로 보완하여, 정확한 위도와 경도 값을 수동으로 입력합니다.
 - 특정 정류소에 대해 수동으로 좌표를 설정하여 데이터를 수정합니다.
- **결과:** 수동으로 입력된 좌표가 포함된 정류소 데이터가 업데이트됩니다.

3. 노선 정보 병합

- **로직:**
 - 버스 노선별 정류소 정보를 포함한 데이터를 불러와, 해당 정류소에 해당하는 노선 정보를 추가합니다.
 - 각 노선에 대한 정류소의 순번 및 고유 ID를 함께 결합하여, 각 버스 정류소가 어떤 노선에 속하는지 정보를 포함합니다.
- **결과:** 버스 노선과 관련된 정류소 정보가 포함된 데이터프레임이 생성됩니다.

4. 승하차 승객 수 정보 병합

- **로직:**
 - 버스 정류소와 관련된 승하차 승객 수 데이터를 불러와, 이를 노선별로 정리합니다.
 - 승차 및 하차 승객 수 데이터를 기반으로, 각 정류소에서의 승차 및 하차 승객 수를 계산합니다.
 - 승차 및 하차 승객 수를 집계하여, 각 정류소의 승객 데이터를 업데이트합니다.
- **결과:** 승객 수 정보가 포함된 정류소 데이터가 생성됩니다.

5. 정렬 및 그룹화

- **로직:**
 - 노선별로 데이터를 그룹화하고, 각 노선에 대해 순번에 따라 데이터를 정렬합니다.
 - 이를 통해 각 노선의 정류소 순서를 정확히 파악할 수 있습니다.
- **결과:** 순번에 따라 정렬된 정류소 목록이 생성됩니다.

6. 맵 시각화

- **로직:**
 - **Folium** 라이브러리를 사용하여 서울시 광진구 지역의 버스 정류소와 노선 정보를 지도에 시각화합니다.

- 각 버스 정류소의 위치를 지도에 마커로 표시하고, 노선 순서에 따라 정류소를 연결하는 선을 그립니다.
- 혼잡도에 따른 색상 변화를 통해, 각 정류소의 승객 수에 따른 혼잡도를 시각적으로 표시합니다.
- **결과:** 광진구 지역의 버스 정류소와 노선 정보가 시각화된 지도 생성, 혼잡도에 따른 색상 변화가 적용된 지도 제공.

전체 흐름 요약

1. 정류소 정보를 기반으로 광진구 내의 버스 정류소 데이터를 필터링하고, 위치 정보를 병합하여 상세 정보를 생성합니다.
2. 수동 좌표 보안을 통해 일부 정류소의 좌표를 정확하게 수정하고, 이 데이터를 반영합니다.
3. 버스 노선 정보를 추가하여 각 정류소가 속한 노선과 순번 정보를 결합합니다.
4. 승하차 승객 수 데이터를 병합하여 각 정류소의 승객 수에 대한 정보를 계산하고 업데이트합니다.
5. 정렬된 데이터를 통해 각 노선별 정류소 순서를 정확하게 파악하고, 이를 기반으로 Folium을 사용하여 시각화합니다.

2. 경사도 계산

1. 경사도 구하는 과정

1. 정류소 데이터를 GeoDataFrame으로 변환

- **로직:** `sorted_df` 데이터를 `GeoDataFrame` 으로 변환하고, 좌표계는 WGS84 (`EPSG:4326`)로 설정합니다.
- **결과:** `sorted_df` 가 `GeoDataFrame`으로 변환되어, 공간적 분석을 위한 기반이 준비됩니다.

2. 정류소에 가장 가까운 고도값 할당

- **로직:**
 - `get_nearest_height` 함수는 각 정류소의 위치에서 가장 가까운 고도 값을 찾아 할당합니다.
 - `elevation_gdf` 의 각 점에서 `sorted_df` 의 각 정류소까지의 거리를 계산하고, 가장 가까운 고도를 선택하여 고도 컬럼에 추가합니다.

- **결과:** 각 정류소에 대해 해당 고도가 추가되어, 경사도 분석을 위한 데이터가 완성됩니다.

3. 정류소 데이터 노선명 및 순번으로 정렬

- **로직:** 정류소를 `노선명` 과 `순번` 기준으로 정렬하여, 노선별로 순서가 제대로 정렬된 데이터프레임을 생성합니다.
- **결과:** 정류소 데이터가 노선별 순서대로 정렬되어, 경사도 분석을 위한 기초 데이터가 준비됩니다.

4. 다음 정류장과의 거리 및 고도차 계산

- **로직:**
 - 각 정류장에 대해 다음 정류장과의 거리(단위: meter)와 고도 차이를 계산합니다.
 - 경사도는 고도 차이를 거리로 나누어 계산하고, 경사도(%) 컬럼에 값을 추가합니다.
 - 거리 계산은 위도와 경도를 사용하여 대략적인 meter 단위로 계산합니다.
- **결과:** `slope_df` 에 각 정류장 간 거리, 고도 차이 및 경사도 정보가 포함된 데이터프레임이 생성됩니다.

5. 경사도 데이터를 기존 정류소 데이터와 결합

- **로직:** `join_df` 를 생성하여 경사도(%) 값을 기존 `sorted_df` 에 조인합니다.
- **결과:** `sorted_df_total` 데이터프레임에 경사도 정보가 추가되어, 각 정류소의 경사도 데이터가 결합된 최종 데이터프레임이 만들어집니다.

경사도 구하는 과정:

- 정류소 데이터를 공간적 분석을 위한 `GeoDataFrame` 으로 변환하고, 각 정류소에 가장 가까운 고도를 할당하여 고도 정보를 추가합니다.
- 거리 및 고도 차이를 계산하여 경사도 정보를 추출하고, 이를 기존 데이터에 결합하여 최종적으로 경사도 정보가 포함된 데이터프레임을 만듭니다.

3. 결빙 위험도

1. 도로빙결 관측 횟수 계산

- **로직:** 도로빙결 위험이 발생한 지역들을 필터링하여, 각 지역(`serial` , `Latitude` , `Longitude`)에서의 도로빙결 관측 횟수를 계산합니다.

- **결과:** 각 지역에서 발생한 도로빙결 위험의 발생 횟수를 담은 집계 데이터 생성.

2. 행정 구역 데이터와 고도 데이터 불러오기 및 전처리

- **로직:**
 - 행정 구역 및 고도 데이터를 각각 로드하고, 이를 동일한 좌표계(`EPSG:4326`, WGS 84)로 변환합니다.
 - 로드한 행정 구역 데이터에서 광진구만 필터링합니다.
 - 고도 데이터를 불러와 해당 지역에서의 고도가 30m 이상인 영역만 필터링합니다.
- **결과:** 광진구 내에서 고도 30m 이상인 지역들의 경계를 포함한 데이터 생성.

3. 위험 지역 데이터 포인트 변환

- **로직:** 위험 지역 데이터에서 주어진 경도 및 위도 값을 사용하여 각 위험 지역을 점 (`Point`)으로 변환합니다.
- **결과:** 위험 지역들을 점 형태로 변환한 데이터를 생성하여, 공간적 위치를 명확히 표현합니다.

4. 위험 지역과 고도 데이터를 필터링 및 결합

- **로직:** 고도가 30m 이상인 지역들과 위험 지역 데이터를 비교하여, 특정 범위 내에 위치한 위험 지역을 필터링하려는 의도였으나 이 부분은 주석 처리되어 실행되지 않았습니다.
- **결과:** 이 작업은 현재는 수행되지 않지만, 실제로는 위험 지역과 고도 데이터를 비교해 고도 30m 이상인 지역 내에 있는 위험 지역을 추출할 수 있었을 것입니다.

5. 맵 생성 및 시각화

- **로직:**
 - `Folium` 라이브러리를 사용하여 지도 시각화를 준비합니다.
 - 광진구의 경계를 지도에 추가하고, 이를 중심으로 맵을 설정하여 `zoom_start=13` 로 줌 수준을 지정합니다.
- **결과:** 광진구의 경계를 지도에 시각화하여 해당 지역을 지도 상에서 확인할 수 있도록 했습니다.

6. 시각화

- 위험 지역 색상 매핑 및 마커 추가

- 로직:

- `Risk_Level` 컬럼을 기준으로 위험 레벨에 색상을 매핑하여 마커를 지도에 추가합니다.
 - 위험 지역의 각 지점에 삼각형 마커를 추가하고, 툴팁을 통해 `Risk_Level` 을 표시합니다.

- 결과: 위험 레벨에 따라 색상이 다른 삼각형 마커가 지도에 추가되어, 위험 지역을 시각적으로 확인할 수 있습니다.

- 버스 정류소 및 노선 연결 시각화

- 로직:

- 각 노선별로 정류소들을 순서대로 연결하는 `PolyLine` 을 추가하여, 버스 노선 경로를 시각화합니다.
 - 방향 화살표를 각 경로의 중간 지점에 추가하여, 노선의 방향을 명확하게 표시합니다.

- 결과: 각 노선의 경로가 파란색 선으로 표시되고, 방향을 나타내는 화살표가 추가됩니다.

- 버스 정류소 팝업 추가

- 로직: 각 정류소에 팝업을 추가하여, 정류소명, 승차/하차 승객 수, 경사도를 표시합니다.

- `folium.CircleMarker` 를 사용하여 각 정류소를 빨간색 원으로 표시합니다.

- 결과: 각 정류소에 대해 승차 및 하차 승객 수, 경사도 등의 정보가 팝업으로 표시됩니다.

- 관측 지점 색상 매핑 및 원 마커 추가

- 로직:

- 각 관측 지점에 대해 관측 횟수(`count`)를 정규화하여 색상을 매핑하고, `folium.Circle` 로 원 마커를 추가합니다.
 - 원의 색상은 관측 횟수에 비례하여, 색상 지도가 시각적으로 적용됩니다.

- 결과: 각 관측 지점이 원 마커로 지도에 추가되어, 관측 횟수에 따른 색상 변화가 나타납니다.

- 고도 30m 이상인 지역 표시

- **로직:**

- 고도가 30m 이상인 지역의 `LineString` 데이터를 회색 선으로 지도에 표시합니다.
- 각 고도 구간에 대한 툴팁을 추가하여 고도를 확인할 수 있습니다.

- **결과:** 고도 30m 이상의 지역이 회색 선으로 지도에 시각화되어, 해당 지역을 식별할 수 있습니다.

- **열선 위치 표시**

- **로직:** `folium.PolyLine` 을 사용하여 열선 경로를 초록색 선으로 지도에 추가합니다.
- **결과:** 열선 설치 위치가 초록색 선으로 지도에 표시되어, 열선 경로를 확인할 수 있습니다.

- **최종 지도 저장**

- **로직:** `folium.Map` 객체를 `bus_map.html` 파일로 저장하여, 웹에서 지도를 시각적으로 확인할 수 있게 합니다.
- **결과:** 생성된 지도는 `bus_map.html` 파일로 저장되어, 웹 브라우저에서 시각화된 내용을 확인할 수 있습니다.

시각화 과정:

- 위험 지역의 색상 매핑, 버스 노선의 연결 및 방향 표시, 정류소 팝업 정보 등을 지도에 시각화합니다.
- 관측 지점, 고도 30m 이상 지역, 열선 경로 등을 지도에 시각화하여, 다양한 정보를 종합적으로 보여줍니다.
- 최종적으로 생성된 지도는 HTML 파일로 저장되어 웹에서 확인할 수 있게 됩니다.

최종적으로 구할 수 있는 것

1. **유동인구 분석:** 버스 승하차 데이터를 바탕으로 도로 구간별 유동인구를 계산하여, 사람들이 많이 이용하는 도로 구간에 열선 설치가 우선될 수 있도록 합니다.
2. **경사도 분석:** 각 도로 구간의 경사도를 계산하여 경사가 심한 도로는 결빙 위험이 높아지기 때문에 우선 설치 대상으로 선정합니다.
3. **결빙 위험도 분석:** 도로 구간별 결빙 위험도를 계산하여 위험한 도로를 파악하고 열선 설치 우선순위를 정합니다.

이 코드의 목적은 위험 요소를 종합적으로 평가하여 열선 설치가 필요한 도로 구간을 정확하게 도출하는 것입니다. 이를 통해 겨울철 도로 결빙 사고를 예방하고, 교통 안전을 향상시킬 수 있습니다.

전체 흐름 요약

1. 도로빙결 위험 관측 데이터를 기반으로 각 지역에서의 발생 횟수를 집계하여 도로빙결 위험이 발생한 지점들을 파악.
2. 행정 구역 및 고도 데이터를 로드하여 광진구 내에서 고도가 30m 이상인 지역을 추출.
3. 위험 지역 데이터를 점 형식으로 변환하여 공간적 정보를 생성.
4. Folium을 사용하여 광진구 경계를 지도에 추가, 해당 지역을 시각화할 수 있도록 준비.

코드 설명

1. 필요한 라이브러리 импорт

```
python
복사편집
import folium
import geopandas as gpd
import pandas as pd
from shapely.geometry import Point
import matplotlib
import matplotlib.cm as cm
import matplotlib.colors as colors
from shapely.ops import nearest_points
import numpy as np
```

- **folium** : 지도 생성 라이브러리로, **Leaflet.js** 를 기반으로 하여 웹 기반의 인터랙티브 지도 시각화를 생성할 수 있습니다.

- **geopandas** : 지리정보 시스템(GIS) 데이터를 다루기 위한 라이브러리입니다.
GeoDataFrame 을 사용해 공간 데이터를 쉽게 처리할 수 있습니다.
- **pandas** : 데이터 처리 및 분석을 위한 라이브러리입니다.
- **shapely.geometry.Point** : 공간에서 점(Point) 데이터를 다룰 때 사용됩니다.
- **matplotlib** : 데이터 시각화 라이브러리로, 컬러 맵 생성 등 시각적 요소들을 처리할 때 사용됩니다.
- **shapely.ops.nearest_points** : 두 점 또는 두 지오메트리 객체 사이의 거리를 계산하거나, 가장 가까운 지점들을 찾을 때 사용됩니다.
- **numpy** : 수학 및 배열 관련 연산을 위한 라이브러리입니다.

2. 도로빙결 관측 데이터 필터링 및 준비

```
python
복사편집
gfdm_3 = pd.read_csv('gwangjin_freezing_data_month_12.csv')
gfdm_3 = gfdm_3[gfdm_3.freezing_risk.isin([True])]
gfdm_3 = gfdm_3[['serial', 'Latitude', 'Longitude']]
grouped_counts = gfdm_3.groupby(['serial', 'Latitude', 'Longitude']).size().reset_index(name='count')
```

- **gfdm_3 = pd.read_csv(...)** : 도로빙결(혹은 동결) 관측 데이터를 CSV 파일에서 읽어옵니다.
- **gfdm_3[gfdm_3.freezing_risk.isin([True])]** : 'freezing_risk' 열에서 **True** 인 데이터를 필터링 하여 동결 위험이 있는 데이터만 추출합니다.
- **grouped_counts = gfdm_3.groupby(...)** : **serial** , **Latitude** , **Longitude** 를 기준으로 그룹화하여 각 위치별로 관측 횟수를 세고, 그 값을 **count** 라는 새 열로 추가합니다.

3. 행정 경계 및 고도 데이터 로드 및 처리

```
python
복사편집
shapefile_path = 'LARD_ADM_SECT_SGG_11_202502.shx'
gbm_total_hjd = gpd.read_file(shapefile_path, encoding='euc-kr')

elevation_shapefile_path = 'N3L_F001.shx'
gbm_total_elevation = gpd.read_file(elevation_shapefile_path, encoding='e
```

```
uc-kr')
```

- `shapefile_path` : 행정구역 경계 데이터를 담고 있는 `shapefile` 파일의 경로입니다.
- `gbm_total_hjd = gpd.read_file(...)` : 행정구역 경계를 읽어와 `GeoDataFrame` 으로 저장합니다.
- `elevation_shapefile_path` : 고도 데이터를 담고 있는 `shapefile` 파일의 경로입니다.
- `gbm_total_elevation = gpd.read_file(...)` : 고도 데이터를 읽어와 `GeoDataFrame` 으로 저장합니다.

4. 좌표계 설정 및 변환

```
python
복사편집
if gbm_total_hjd.crs is None:
    gbm_total_hjd.set_crs('EPSG:5179', inplace=True)
if gbm_total_elevation.crs is None:
    gbm_total_elevation.set_crs('EPSG:5179', inplace=True)

target_gbm_total_hjd = gbm_total_hjd.to_crs('EPSG:4326')
gbm_total_total_elevation = gbm_total_elevation.to_crs('EPSG:4326')
```

- `set_crs('EPSG:5179')` : `EPSG:5179` 는 한국에서 사용되는 좌표계이므로, `GeoDataFrame` 에 좌표계가 지정되지 않은 경우 해당 좌표계를 설정합니다.
- `to_crs('EPSG:4326')` : 한국 좌표계를 `WGS 84` (GPS 좌표계, `EPSG:4326`)로 변환합니다.

5. 광진구만 필터링

```
python
복사편집
mask = gbm_total_hjd['SGG_NM'] == '광진구'
target_gbm_total_hjd_gwangjin = target_gbm_total_hjd[mask].to_crs('EPSG:4326')
```

- `mask` : `SGG_NM` 열에서 값이 `광진구` 인 행들만 필터링합니다.

- `target_gbm_total_hjd[mask]` : 필터링된 데이터만 `GeoDataFrame` 으로 가져옵니다.

6. 고도 데이터 필터링

```
python
복사편집
gbm_total_total_elevation_gwangjin = gbm_total_total_elevation[
    gbm_total_total_elevation.geometry.intersects(target_gbm_total_hjd_gwa
ngjin.unary_union)
]
```

- `geometry.intersects(...)` : 고도 데이터를 광진구의 행정구역과 교차하는 영역만 필터링합니다.

7. 고도 30m 이상 필터링

```
python
복사편집
absolute_threshold = 30
gbm_total_high_elevation = gbm_total_total_elevation_gwangjin[
    gbm_total_total_elevation_gwangjin['HEIGHT'] >= absolute_threshold
]
```

- `absolute_threshold = 30` : 30미터 이상의 고도를 가진 데이터를 필터링합니다.
- `gbm_total_high_elevation` : 고도 30미터 이상인 데이터를 추출합니다.

8. `GeoDataFrame` 생성 및 좌표 데이터 변환

```
python
복사편집
df['geometry'] = df.apply(lambda row: Point(row['Longitude'], row['Latitude']), axis=1)
risk_gdf = gpd.GeoDataFrame(df, geometry='geometry', crs='EPSG:4326')
```

- `df['geometry']` : 각 행에 대해 `Longitude` , `Latitude` 를 사용하여 `Point` 객체를 생성하고, 이를 `geometry` 열에 저장합니다.
- `risk_gdf = gpd.GeoDataFrame(...)` : `df` 를 `GeoDataFrame` 으로 변환하고, 좌표계는 `EPSG:4326` 로 지정합니다.

9. 반경 100m 이내의 `Risk_Level` 데이터 필터링

```
python
복사편집
filtered_risk_gdf = risk_gdf
```

- 현재 주석처리된 코드는 `Risk_Level` 이 특정한 조건에 맞는 점들만 필터링하는 코드입니다.

10. 정류소 데이터 병합

```
python
복사편집
excel_file_path = r"서울시 정류소현황(2019~2023년).xlsx"
csv_file_path = r"서울시 버스정류소 위치정보.csv"

df_excel = pd.read_excel(excel_file_path)
df_csv = pd.read_csv(csv_file_path, encoding='utf-8')
```

- `df_excel` : 정류소 현황 데이터 엑셀 파일을 읽어옵니다.
- `df_csv` : 정류소 위치 정보 CSV 파일을 읽어옵니다.

다음 섹션에서는 데이터 병합, 좌표 보완, 노선 정보 및 승하차 정보 등을 처리하는 부분을 설명하겠습니다.

4o mini

나의 말:

그래 계속해

ChatGPT의 말:

좋아요, 계속해서 나머지 부분을 설명해드리겠습니다. 이번에는 데이터 병합, 좌표 보완, 그리고 버스 정류소 관련 작업을 다루는 부분입니다.

11. 버스 정류소 데이터 병합

```
python
복사편집
df_merged = pd.merge(df_excel, df_csv, left_on='정류소명', right_on='정류소명', how='inner')
```

- `pd.merge(...)` : `df_excel` 과 `df_csv` 두 데이터프레임을 '정류소명' 열을 기준으로 병합합니다. `how='inner'` 는 두 데이터프레임에 모두 존재하는 '정류소명' 값에 대해서만 병합을 수행하도록 지정합니다.
- 이렇게 병합된 데이터에는 엑셀 파일과 CSV 파일의 정류소 관련 정보가 모두 포함됩니다.

12. 위도 및 경도 좌표 처리

```
python
복사편집
df_merged['geometry'] = df_merged.apply(lambda row: Point(row['Longitude'], row['Latitude']), axis=1)
bus_stops_gdf = gpd.GeoDataFrame(df_merged, geometry='geometry', crs='EPSG:4326')
```

- `df_merged.apply(...)` : `Longitude` 와 `Latitude` 열을 이용해 각 버스 정류소의 좌표를 `Point` 객체로 변환합니다.
- `bus_stops_gdf = gpd.GeoDataFrame(...)` : 변환된 좌표들을 기반으로 `GeoDataFrame` 을 생성합니다. 이렇게 하면 공간적인 처리가 가능해집니다.

13. 도로빙결 위치와 버스 정류소 간의 거리 계산

```
python
복사편집
risk_gdf['nearest_bus_stop'] = risk_gdf.geometry.apply(lambda x: nearest_
```

```
points(x, bus_stops_gdf.unary_union)[1])
```

- `nearest_points(x, bus_stops_gdf.unary_union)` : `risk_gdf` 에 있는 각 점(`x`)과 버스 정류소들 간의 가장 가까운 정류소를 계산합니다. `unary_union` 은 `bus_stops_gdf` 의 모든 정류소를 하나의 큰 지오메트리로 결합한 것입니다.
- `nearest_points(...)[1]` : `nearest_points` 는 두 점 사이의 가장 가까운 위치를 반환하는데, `[1]` 은 두 번째 반환값인 "가장 가까운 버스 정류소"를 의미합니다.
- `risk_gdf['nearest_bus_stop']` : 각 `risk_gdf` 의 점에 대해 가장 가까운 버스 정류소의 좌표를 저장합니다.

14. 위험 레벨에 따른 마커 색상 지정

```
python
복사편집
cmap = cm.get_cmap('YlOrRd') # 색상 맵 (yellow to red)
norm = colors.Normalize(vmin=0, vmax=5) # 위험 수준 범위
```

- `cm.get_cmap('YlOrRd')` : 'YlOrRd'라는 색상 맵을 사용합니다. 이는 위험 수준이 낮을수록 노란색, 높을수록 빨간색으로 색깔을 지정합니다.
- `colors.Normalize(vmin=0, vmax=5)` : 위험 수준을 0부터 5까지의 범위로 정규화합니다. 이 범위 내에서 색상이 적용됩니다.

15. 버스 정류소 지도에 마커 추가

```
python
복사편집
m = folium.Map(location=[latitude, longitude], zoom_start=12)
for idx, row in bus_stops_gdf.iterrows():
    folium.CircleMarker(location=[row['Latitude'], row['Longitude']], radius=
5, color=cmap(norm(row['Risk_Level'])), fill=True).add_to(m)
```

- `folium.Map(...)` : 지도 중심 좌표(`latitude` , `longitude`)와 초기 확대 레벨(`zoom_start=12`)을 지정하여 `folium.Map` 객체를 생성합니다.

- `for idx, row in bus_stops_gdf.iterrows()` : `bus_stops_gdf` 에서 각 정류소에 대해 반복문을 돌며, 각 정류소에 대해 마커를 추가합니다.
- `folium.CircleMarker(...)` : 각 정류소 위치에 원형 마커를 추가하며, `color=cmap(norm(...))` 를 통해 해당 정류소의 위험 수준에 따라 색상을 다르게 합니다.

16. 위험 레벨에 따른 마커 및 지도에 표시

```
python
복사편집
for idx, row in risk_gdf.iterrows():
    nearest_stop = row['nearest_bus_stop']
    folium.Marker(location=[nearest_stop.y, nearest_stop.x], icon=folium.Icon(
n(color='red')).add_to(m)
```

- `for idx, row in risk_gdf.iterrows()` : 각 `risk_gdf` 의 데이터 행을 반복하면서, 위험 수준이 높은 지역에 대해 마커를 추가합니다.
- `folium.Marker(...)` : 각 위험 지역의 가장 가까운 버스 정류소에 마커를 추가합니다. 마커는 빨간색으로 표시됩니다.

17. 지도에 경로 추가 및 스타일 지정

```
python
복사편집
for _, row in gbm_total_high_elevation.iterrows():
    folium.PolyLine(
        locations=[(lat1, lon1), (lat2, lon2)],
        color="blue",
        weight=3,
        opacity=0.7
    ).add_to(m)
```

- `folium.PolyLine(...)` : 고도가 30미터 이상인 지점 간에 경로를 그리기 위한 `PolyLine` 을 추가합니다. `color="blue"` 는 경로 색상을 파란색으로 설정합니다.

18. 지도 저장

```
python  
복사편집  
m.save("map.html")
```

- `m.save(...)` : 완성된 지도를 HTML 파일로 저장하여 웹에서 확인할 수 있도록 합니다. 생성된 `map.html` 파일을 브라우저에서 열어 지도 상의 마커와 경로를 확인할 수 있습니다.